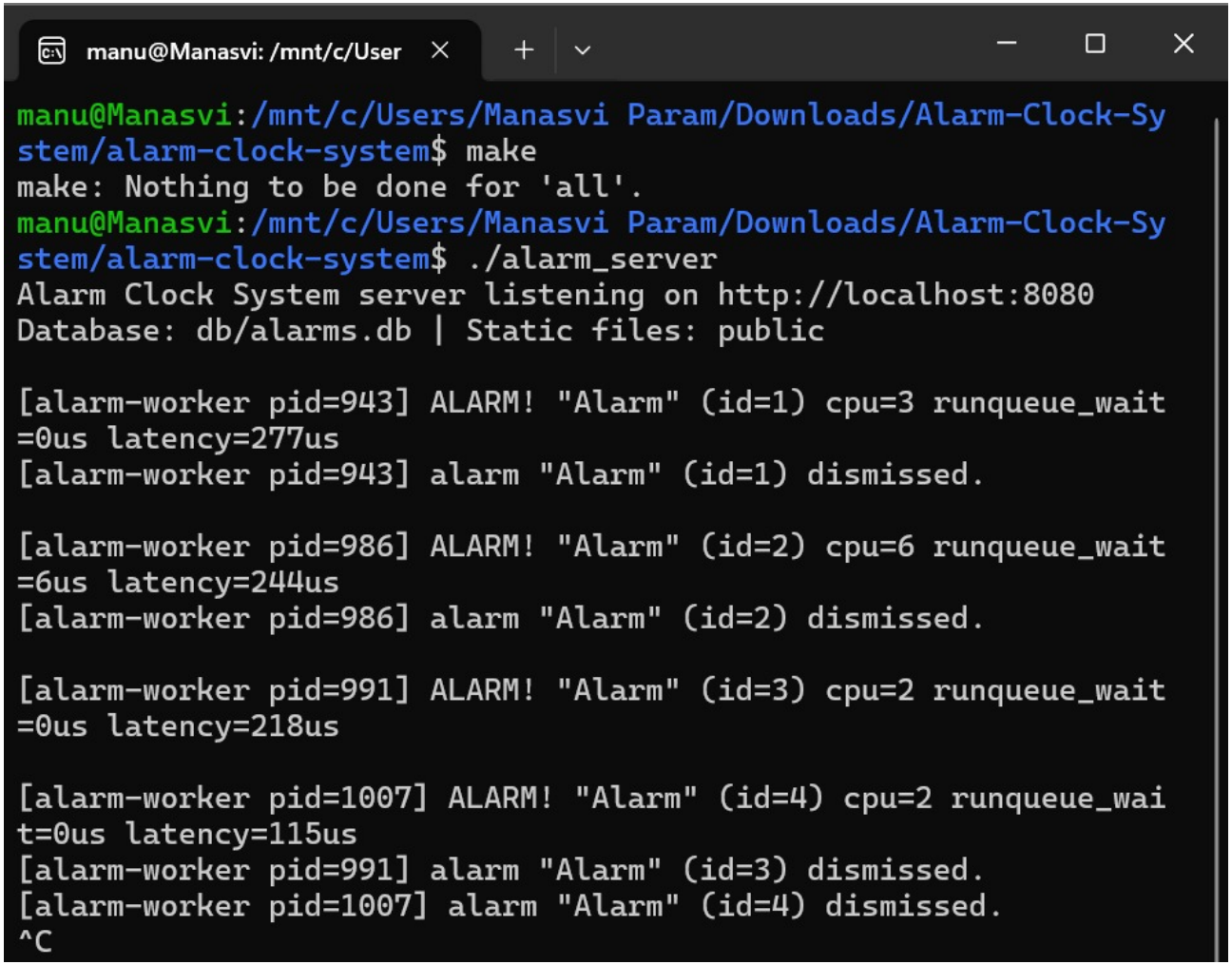
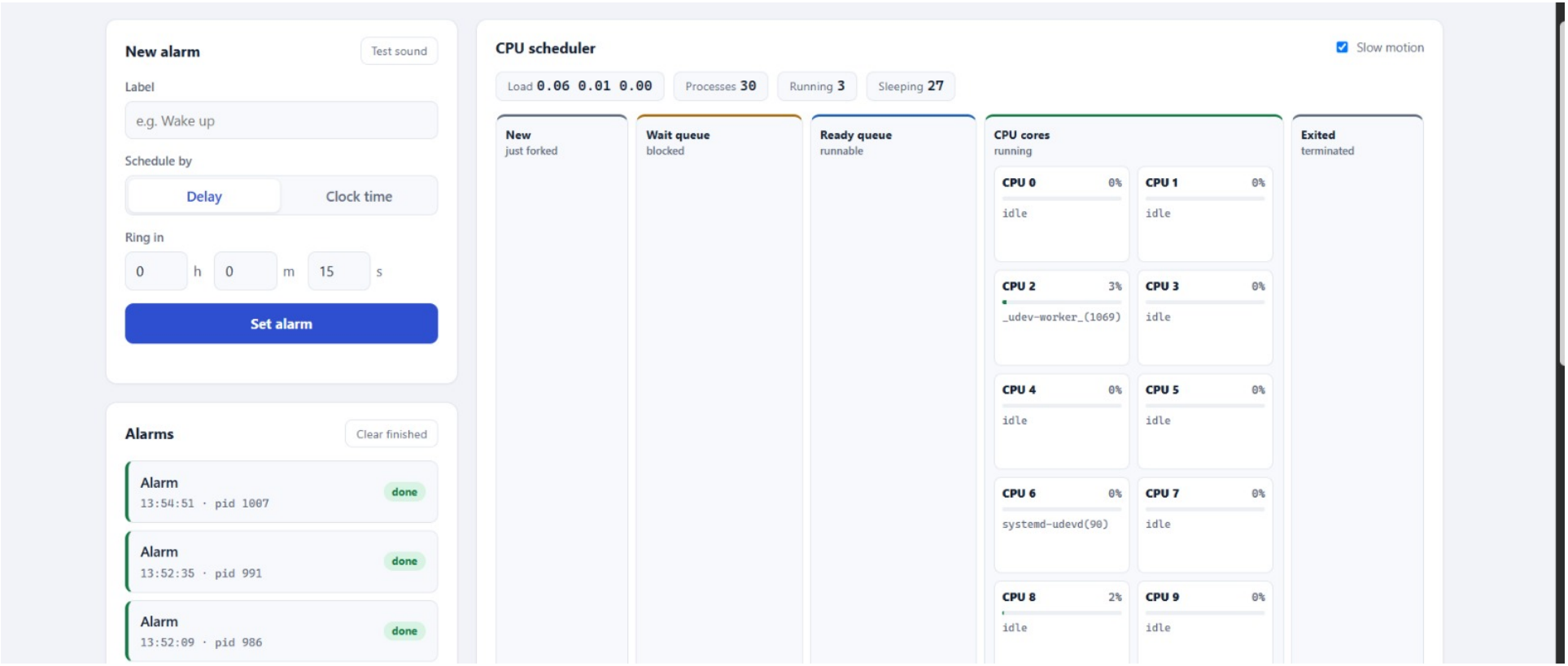
1. Build and start the server

The terminal shows the project being built with **make** and the C alarm server starting on **http://localhost:8080**, with SQLite used to store alarm data.



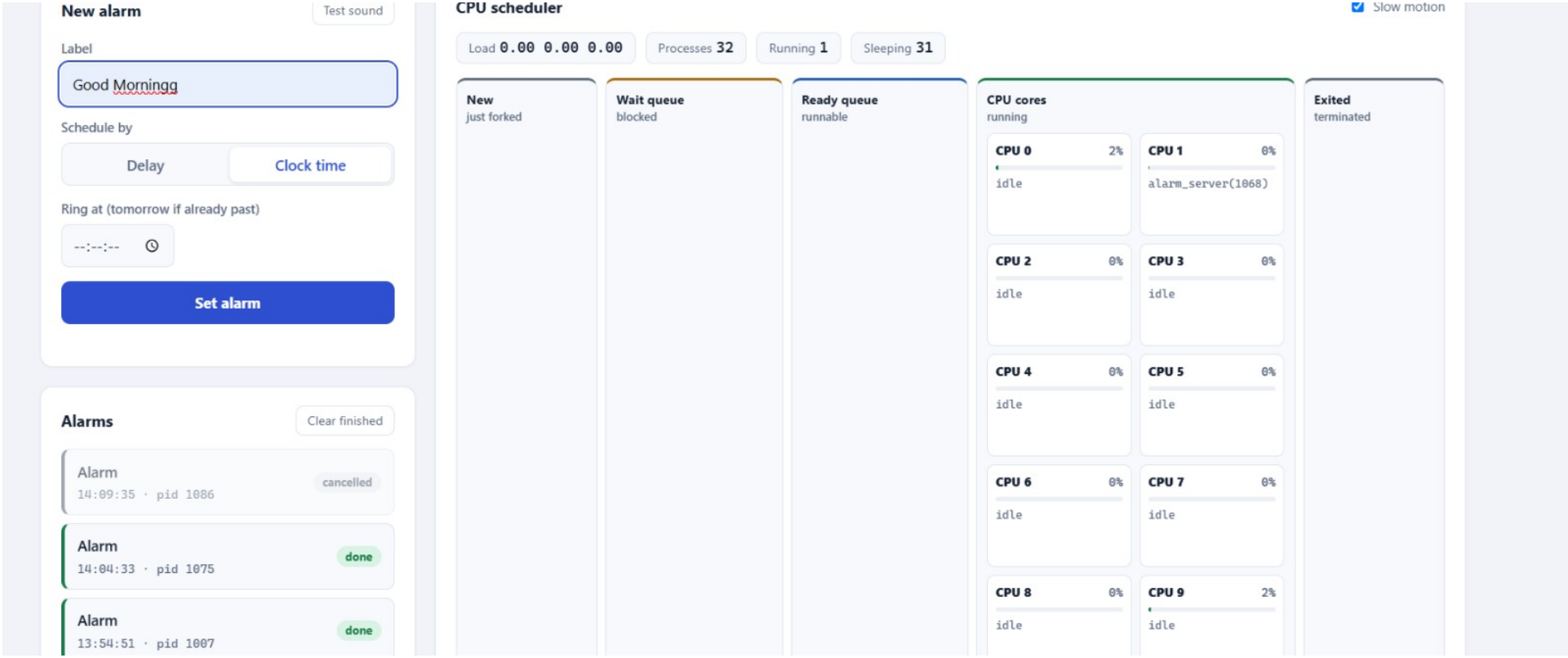
2. Open the Alarm Clock System interface

The home page displays the alarm form and live CPU scheduler, ready for the user to create a new alarm using a delay.



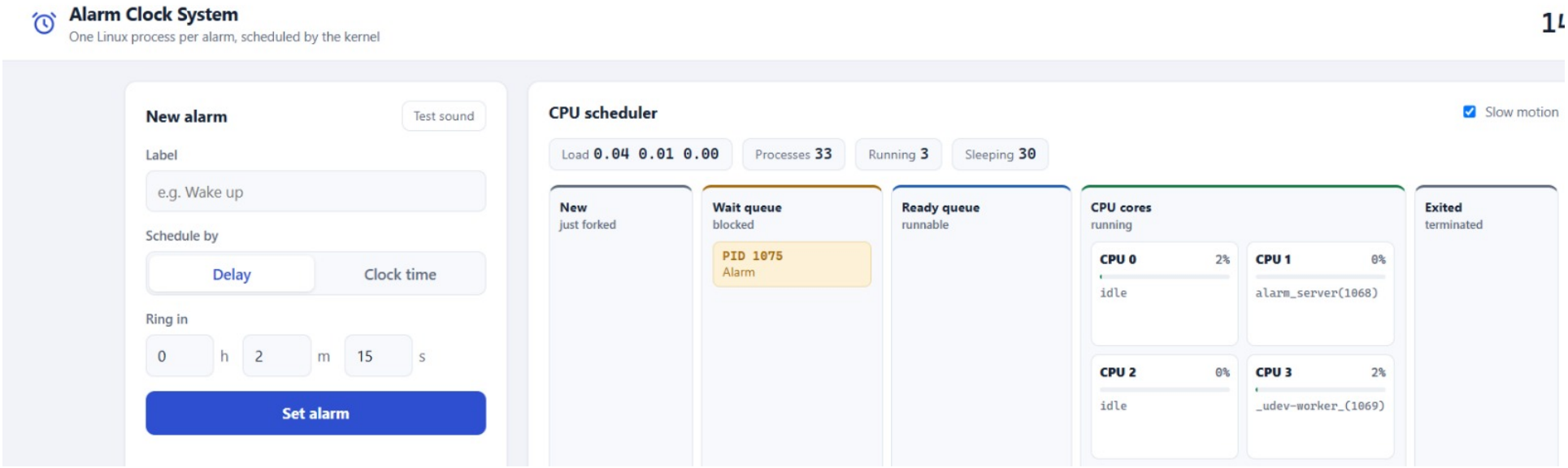
3. Configure an alarm using clock time

The user enters the label **Good Morning**, selects **Clock time**, chooses the ring time, and is ready to press **Set alarm**.



4. Observe the alarm worker in the scheduler

After an alarm is created, the server uses **fork()** to create a separate worker process; the scheduler view shows that process in the wait queue and displays CPU activity.



5. Monitor process states and scheduler events

The live process table and event log show the worker moving through states such as **NEW** → **RUNNING** → **WAITING** → **READY**, along with its PID, CPU core, and timing details.

Chips are real alarm processes. Moves come from events each worker logs itself (real core and timestamps); slow motion only stretches the playback, the log shows true timings. The process table below is live, so it may already show the next state.

Processes live from /proc

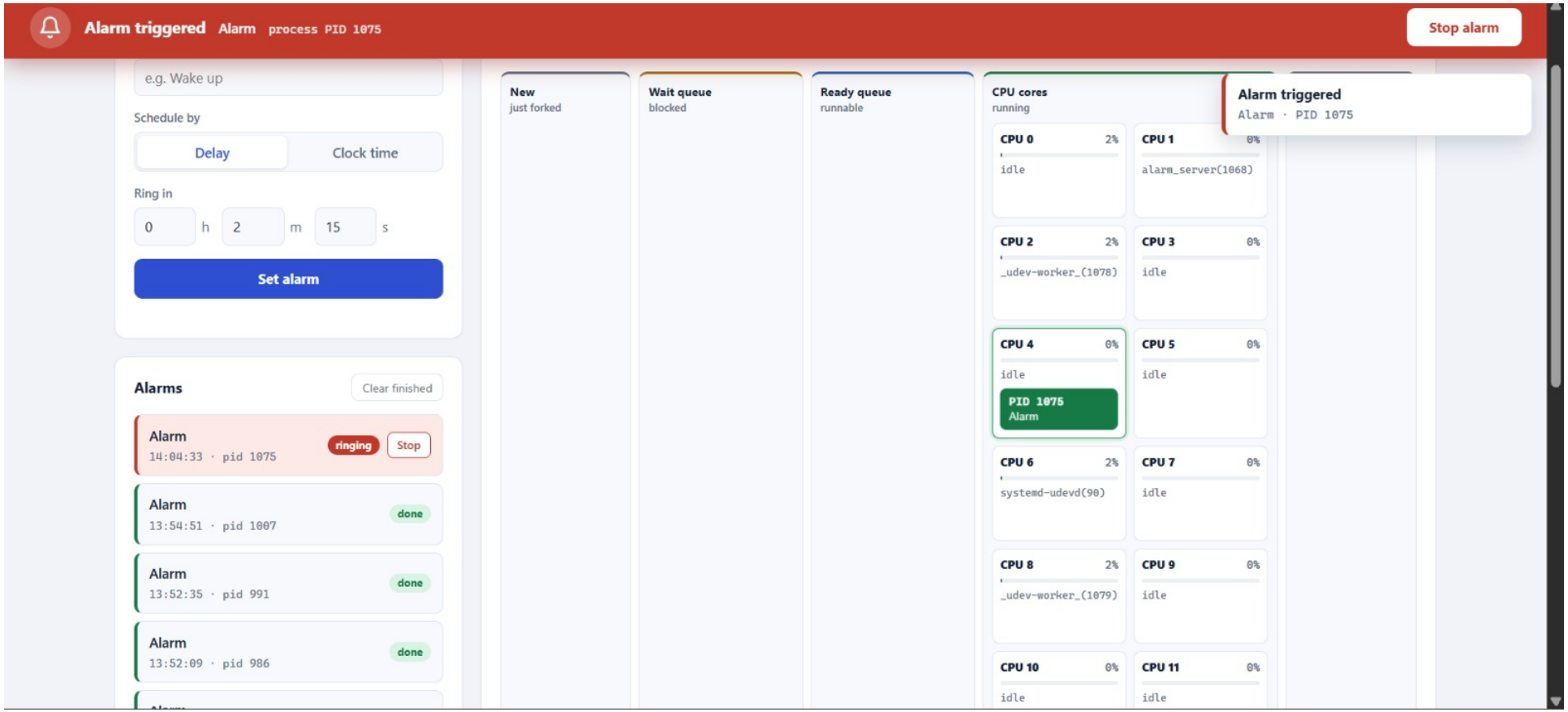
PID	Process	State	Core	CPU time	Context switches
1068	alarm_server (server)	Running	CPU 5	710 ms	4367 voluntary / 8 forced
90	systemd-udevd	Running	CPU 1	7900 ms	24457 voluntary / 148 forced
1076	_udev-worker_	Running	CPU 9	70 ms	144 voluntary / 0 forced
1078	_udev-worker_	Running	CPU 2	1340 ms	1690 voluntary / 9 forced

Event log

```
14:02:18.732  NEW      pid 1075      fork() created this process (parent pid 1068)
14:02:18.756  RUNNING  pid 1075 cpu4 First time on a CPU: installing SIGALRM/SIGUSR1/SIGUSR2 handlers, arming alarm()
14:02:18.764  WAITING  pid 1075 cpu4 sigsuspend(): blocked, removed from the run queue, using 0% CPU until SIGALRM in 135s
14:04:33.764  READY    pid 1075 cpu4 Timer expired, SIGALRM sent: kernel moved the process from the wait queue to the run queue
                                     (timer scheduled for 1791189273)
14:04:33.764  RUNNING  pid 1075 cpu4 Scheduler dispatched it on CPU 4 after 53 us in the run queue; SIGALRM handler ran (timer
                                     latency 273 us)
14:04:33.857  RUNNING  pid 1075 cpu4 notify-send unavailable (install libnotify-bin); notified via terminal and web UI only
14:04:33.864  WAITING  pid 1075 cpu4 Ringing: blocked in sigsuspend() again, waiting for dismissal (SIGUSR2)
14:04:51.621  RUNNING  pid 1075 cpu6 Dismiss signal received: alarm stopped by the user
14:04:51.637  EXITED   pid 1075 cpu6 exit(0): process terminated, parent reaps it via SIGCHLD
```

6. Alarm triggers and waits for dismissal

When the timer expires, the worker receives **SIGALRM**, the interface displays the red alarm banner, and the user can stop the ringing alarm.



7. Confirm completed alarms

After the user dismisses the alarm, the worker exits and the Alarms panel marks it as **done**; the terminal also records the trigger and dismissal events.

